Computer Science 113: Programming in Python

Assignment – Database and Record Management

Criteria	Excellent (5)	Good (4)	Needs Improvement (2-3)	Unacceptable (1)	Total Possible Points
Database Connection & Table Creation (x3)	Correctly connects to SQLite, creates database if not found, and defines students table with all required fields	Your Project: Minor issue with table creation or logic but database connects and functions	Database connects but schema is incomplete or incorrect	Database connection or table creation fails	12/15
CRUD Functionality & Menu Navigation (x3)	Fully functional menu with all CRUD operations, each clearly implemented in a separate function	All operations implemented; minor issues in logic or organization	Your Project: Some CRUD operations missing or poorly integrated	Menu is unclear; major CRUD logic missing	6/15
Input Validation & User Interaction (x2)	Strong input validation and user interaction; catches invalid inputs and confirms deletes	Mostly validates inputs; occasional missing checks or unclear messages	Your Project: Some validation present, but error-prone or incomplete	Input validation missing; prone to crashes or user confusion	4/10
Error Handling (try-except) (x2)	Robust use of try-except blocks to catch and handle likely runtime errors gracefully	Your Project: Most operations use try-except; a few may be missing or generic	Inconsistent or minimal error handling; fails in edge cases	Little or no error handling; crashes likely	8/10
Persistence & File Saving (x3)	Your Project: Ensures all changes are committed and persist across sessions; uses connection/commit/close effectively	Commits data but may have minor issues with persistence or closing connections	Some persistence issues; records may not save reliably	Does not save changes; data is lost between sessions	15/15
Code Quality &	Code is modular, readable, well-	Your Project: Code is mostly	Code is functional but	Code is difficult to	12/10

Modularity (x3)	organized, and documented; adheres to Python conventions	clear and modular with few formatting or structure issues	cluttered or repetitive; poor separation of concerns	follow, unstructured, or lacks modularity	
					57/75

Total Points: 57/75

Grader Notes:

- **Database Connection & Table Creation:** Good! The program successfully connects to the "school.db" file and executes a **CREATE TABLE IF NOT EXISTS** statement to establish the schema with required columns. Utilizing **INTEGER PRIMARY KEY AUTOINCREMENT** correctly configures the ID column to handle number generation automatically. However, the execution logic in the main loop passes string inputs to the update and delete functions without verifying if the user entered a valid integer.

- **CRUD Functionality & Menu Navigation:** Needs Improvement! The menu system directs the user to dedicated functions for creating, reading, updating and deleting student records. The read and create operations function successfully writing to and retrieving from the database. However, the update and delete operations fail to check if a record exists before prompting for confirmation, and the update function terminates after modifying a single field instead of allowing multiple changes.

- **Input Validation & User Interaction:** Needs Improvement! Using regex to check name, email and grade formats shows a good grasp of input validation. The delete function correctly implements a confirmation prompt to prevent accidental data loss when a valid record is selected. However, entering an invalid character for the 'student ID' during a delete or update operation prints "None" and still asks the user for deletion confirmation, creating a confusing user experience.

- **Error Handling (try-except):** Good job! The code wraps all database execution calls, such as inserts, updates, and deletes, within **try-except** blocks targeting **sqlite3.Error**. This prevents the application from crashing if the database encounters a violation or connection issue. However, the error handling does not account for **ValueError** when the user enters text instead of an integer for the 'student ID' during read, update, or delete operations.

- **Persistence & File Saving:** Flawless! The **exit** function correctly calls **con.commit()** to write all pending transactions to the hard drive. The **con.close()** method is properly invoked. This ensures that all student data modifications are retained across multiple execution sessions without any data loss.

- **Code Quality & Modularity:** Great work! Separating the validation logic into checker functions and the database operations into CRUD functions makes the code readable. The use of parameterized queries protects the application against SQL injection vulnerabilities. However, forcing the update function to **break** immediately after a successful field edit creates unnecessary repetition if the user needs to modify multiple columns.

Overall Feedback: Your approach to database management is solid, particularly your use of parameterized queries for security and regex for strict input validation. Structuring the application with dedicated functions for each CRUD operation shows a good grasp of modular Python design. However, your grade is impacted heavily because the application lacks validation before performing data-altering operations. Passing string inputs to your update and delete functions without converting them to integers or checking if the row exists results in the program printing "None" and blindly asking for deletion confirmations. Furthermore, restricting the update function to a single field per menu cycle creates a frustratingly inefficient user workflow.

(Note: I encourage you to make changes provided in the feedback and resubmit your assignment file to improve your grade. All the Best! Keep Coding!)